

PDS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 04 -

CONSTRUINDO APIS COM GRAPHQL

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA? 10

REFERÊNCIAS 11

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Boas-vindas à aula de Construção de APIs com GraphQL, onde vamos entender a parte técnica e prática do Graphql. O foco será desenvolver uma compreensão sobre estruturação de esquemas, implementação de resolvers e integração com Java.

Começaremos entendendo o papel central do Schema Definition Language (SDL), que define a estrutura e as regras do que nossa API GraphQL pode oferecer. Em seguida, exploraremos os resolvers, que são os conectores entre as consultas definidas no esquema e a lógica de negócios que manipula os dados.

Vamos integrar essa lógica com o ecossistema Java, utilizando bibliotecas populares como GraphQL Java para criar APIs escaláveis e eficientes, seguindo boas práticas de desenvolvimento.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Agora que já exploramos os conceitos fundamentais de GraphQL, é hora de partir para a prática na nossa aula. Aqui, vamos aplicar o que aprendemos, construindo passo a passo uma API funcional e integrada com Java.

Iniciaremos com a criação do esquema GraphQL, definindo queries e mutations. Em seguida, configuraremos os resolvers para conectar as definições do esquema à lógica de manipulação de dados.

Na parte de integração, utilizaremos ferramentas do ecossistema Java, como o GraphQL Java, para conectar nossa API ao backend, garantindo uma comunicação com o banco de dados ou serviços externos. Além disso, abordaremos como testar e validar nossas APIs, utilizando ferramentas como o GraphiQL para testar o funcionamento.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Vamos aprender mais sobre os resolvers, que são a ponte entre o esquema GraphQL e a lógica de negócios da aplicação. Eles atuam como os "intérpretes" das operações definidas no esquema, garantindo que as consultas feitas pelos clientes sejam traduzidas em ações concretas no backend. Para cada campo solicitado em uma query ou mutation, o GraphQL invoca o resolver correspondente, que então recupera ou manipula os dados necessários.

Imagine uma consulta que pede informações detalhadas de um usuário. O GraphQL, ao processar essa requisição, direciona cada parte da consulta para o resolver apropriado. Por exemplo, se o campo solicitado for user, o resolver associado será responsável por buscar os dados desse usuário em um banco de dados ou outro sistema. Se, dentro desse user, o cliente solicitar também os campos name e email, os resolvers para esses campos serão chamados. Essa hierarquia de execução permite grande flexibilidade, mas exige cuidado para evitar problemas de desempenho, como o famoso "N+1", onde múltiplas consultas redundantes são feitas ao banco de dados.

Os resolvers, por natureza, são assíncronos, já que muitas operações no back-end — como acessar um banco de dados ou fazer uma requisição HTTP — levam tempo para serem concluídas. Isso significa que é possível lidar com operações complexas sem bloquear o processamento de outras partes da API. Porém, essa flexibilidade também exige que pessoas desenvolvedoras tenham atenção ao gerenciar as dependências e ao desempenho da aplicação.

Como boas práticas, os resolvers devem ser leves, servindo apenas para orquestrar o fluxo de dados. A lógica de negócios mais densa deve ser encapsulada em serviços ou repositórios separados, garantindo um código mais limpo e reutilizável. Além disso, ferramentas como o DataLoader podem ser usadas para otimizar o desempenho, agrupando solicitações e reduzindo o número de chamadas ao banco de dados.

Os resolvers não são apenas "mais uma camada" na arquitetura de uma API GraphQL. Eles são a peça que transforma um simples esquema em uma API funcional e eficiente, conectando as necessidades do cliente à complexidade do backend.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Falaremos agora sobre boas práticas e o que pode ajudar com problemas de “N+1”, os DataLoaders. Os DataLoaders são ferramentas essenciais para otimizar o desempenho de APIs GraphQL, especialmente em situações onde múltiplas consultas a fontes de dados poderiam resultar em operações redundantes e lentas. Eles funcionam como uma espécie de intermediário inteligente entre os resolvers e as operações de acesso a dados, agrupando e deduplicando solicitações para evitar o problema de "N+1".

Imagine que temos uma consulta em nossa API que retorna uma lista de pedidos, onde cada pedido contém informações sobre o cliente associado. Em uma abordagem sem DataLoader, cada pedido pode desencadear uma consulta separada ao banco de dados para buscar os dados do cliente, resultando em múltiplas operações desnecessárias. Se a lista contiver 50 pedidos, teremos 50 chamadas ao banco para buscar os clientes, mesmo que muitos desses pedidos pertençam ao mesmo cliente.

O DataLoader entra exatamente aqui, atuando como um agrupador. Ele coleta todas as solicitações feitas para o mesmo campo em um intervalo de tempo curto e as processa em lote, enviando uma única consulta ao banco de dados com todas as informações necessárias. Ao mesmo tempo, ele elimina duplicatas, garantindo que apenas uma solicitação seja feita para cada cliente único.

Na prática, isso significa que o DataLoader funciona em duas etapas: primeiro, ele registra as solicitações feitas por diferentes resolvers; depois, processa essas solicitações em lote e distribui as respostas de maneira eficiente. Tudo isso ocorre de forma transparente, sem alterar a lógica principal dos resolvers, o que facilita sua integração em APIs existentes.

O resultado é uma API muito mais eficiente, capaz de lidar com grandes volumes de dados e consultas complexas sem comprometer o desempenho. Os DataLoaders não apenas reduzem a carga no banco de dados, mas também tornam a experiência do cliente mais rápida e responsiva, ajudando a atender às demandas de aplicações e deixar tudo mais escalável.

Outra feature muito apreciada no Graphql é a das subscriptions. As subscriptions no GraphQL são uma poderosa funcionalidade que permite aos clientes receberem atualizações em tempo real sempre que determinados eventos ocorrem

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

no servidor. Diferente das queries e mutations, que funcionam em um modelo de requisição e resposta, as subscriptions estabelecem uma conexão persistente entre o cliente e o servidor, criando um fluxo contínuo de dados.

Pense nas subscriptions como uma espécie de "assinatura" para eventos. Quando o cliente executa uma subscription, ele diz ao servidor: "Quero ser notificado sempre que algo específico acontecer." Por exemplo, imagine um sistema de chat. Com subscriptions, um cliente pode receber novas mensagens assim que forem enviadas, sem precisar ficar requisitando manualmente essas informações.

Tecnicamente, isso é implementado por meio de protocolos que permitem comunicação bidirecional, como o WebSocket. Quando uma subscription é ativada, o servidor mantém aberta uma conexão com o cliente e envia dados sempre que a condição da subscription for satisfeita. Esses dados são enviados no mesmo formato definido pelo esquema GraphQL, garantindo consistência com o resto da API.

No esquema GraphQL, uma subscription pode ser definida assim:

```
type Subscription {
  messageAdded: Message
}
type Message {
  id: ID!
  content: String!
  sender: String!
}
```

Código-fonte 1 – Exemplo de schema com subscription em Graphql
Fonte: elaborado pelo autor (2024)

Nesse exemplo, sempre que uma nova mensagem for adicionada, o servidor notificará os clientes que tiverem uma subscription ativa para messageAdded.

No back-end, a implementação das subscriptions exige o uso de um mecanismo de publicação e assinatura, também conhecido como Pub/Sub. Quando um evento ocorre, ele é publicado e todos os clientes inscritos são notificados. Essa abordagem é particularmente útil para cenários como notificações em tempo real, atualizações de status, sistemas de monitoramento e muito mais.

As subscriptions trazem um novo nível de interatividade para APIs, tornando-as ideais para aplicações que demandam dados dinâmicos e atualizações instantâneas. Ao mesmo tempo, sua implementação requer atenção a detalhes como

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

escalabilidade, gerenciamento de conexões e segurança, para garantir que a API continue eficiente e confiável em produção.

O GraphQL é uma especificação extremamente versátil e intencionalmente aberta a diferentes abordagens. Isso significa que há espaço para adaptar sua implementação às necessidades específicas do projeto, mas também exige o uso de algumas boas práticas para lidar com desafios comuns, como controle de versão, autorização e paginação.

Ao contrário das APIs REST, que dependem de múltiplos endpoints, o GraphQL centraliza todas as capacidades do serviço em um único endpoint HTTP. Isso facilita o gerenciamento, mas requer atenção ao uso de ferramentas como o GraphiQL, que ajudam a explorar e testar o esquema com eficiência. Além disso, o uso de JSON como formato de resposta, combinado com compressão GZIP, equilibra a familiaridade para pessoas desenvolvedoras e a eficiência no tráfego de dados, garantindo respostas leves e rápidas.

Sobre controle de versão, o GraphQL propõe uma abordagem inovadora: em vez de criar versões incrementais, ele permite a evolução contínua do esquema. Isso é possível porque o cliente sempre solicita explicitamente os dados que deseja. Assim, novos campos e tipos podem ser adicionados sem quebrar contratos existentes, promovendo APIs mais simples de manter.

Outro conceito essencial é o da nulidade. Por padrão, todos os campos em GraphQL são anuláveis, permitindo maior resiliência em casos de erro, como falhas de banco de dados ou regras de autorização que impedem o acesso a determinados campos. Quando o comportamento de "null" não for apropriado, os tipos não nulos podem garantir que um campo sempre tenha valor, oferecendo mais previsibilidade aos clientes.

Já no que diz respeito à paginação, o GraphQL oferece flexibilidade, permitindo que listas extensas sejam divididas em partes menores usando argumentos como first e after. O padrão Connections, amplamente utilizado, é reconhecido por ferramentas como o Relay, que tornam a paginação no cliente muito mais eficiente e consistente.

Por fim, práticas como armazenamento em lote e cache no lado do servidor são indispensáveis para evitar a repetição desnecessária de operações e melhorar o desempenho da API. Ferramentas como o DataLoader agrupam solicitações e as

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

processam de forma otimizada, reduzindo significativamente o impacto de consultas redundantes em bancos de dados ou microsserviços.

Essas práticas, embora opinativas, são baseadas em experiências reais com o GraphQL e ajudam a construir APIs mais escaláveis, eficientes e alinhadas às demandas de aplicações que construímos no dia a dia.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o universo do GraphQL e aprendemos como construir APIs modernas e eficientes. Começamos entendendo a importância de estruturar um esquema bem definido, usando o Schema Definition Language (SDL) para expressar as capacidades da API. Isso incluiu criar queries, mutations e até subscriptions, que permitem interatividade em tempo real.

Aprofundamos no papel dos resolvers, que conectam o esquema à lógica de negócios, e discutimos as melhores práticas para garantir performance e manutenibilidade. Aprendemos como evitar problemas de desempenho, como o famoso "N+1", utilizando ferramentas como o DataLoader para agrupar e otimizar consultas ao back-end.

Também abordamos a integração prática com Java, utilizando bibliotecas populares como o GraphQL Java, para implementar APIs robustas e alinhadas às demandas de aplicações reais. Você viu como testar, depurar e validar suas APIs usando ferramentas como o GraphiQL.

Além disso, discutimos conceitos fundamentais de boas práticas em GraphQL, como a evolução contínua do esquema, a importância de projetar para nulidade e as estratégias de paginação para lidar com listas extensas de dados.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

GRAPHQL**. Introduction to GraphQL**. 2024. Disponível em: <https://graphql.org/learn/>. Acesso em: 10 dez. 2024.

GRAPHQL**. Queries and Mutations**. 2024. Disponível em: <https://graphql.org/learn/queries/>. Acesso em: 10 dez. 2024.

GRAPHQL**. Schemas and Types**. 2024. Disponível em: <https://graphql.org/learn/schema/>. Acesso em: 10 dez. 2024.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

GraphQL. API Flexível. Consultas Personalizadas. Schema Definition Language (SDL). Resolvers. Eficiência em Dados. Overfetching. Underfetching. Microsserviços. GraphQL Server. GraphiQL. Integração Cliente-Servidor. Tipos Fortemente Tipados. Fragmentos. Diretivas. Subscriptions. Mutations. Query Language. N+1 Problem. DataLoader. Documentação Dinâmica. Ferramentas Dev. Cloud-Native. APIs Escaláveis. Apollo. Relay. Prisma. Integração Fullstack. Depuração. Versionamento Suave. Segurança em APIs.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com